

Data Structure and Programming, Spring 2025
programming assignment #2
Finding the Largest Land for Building

Pei-Yuan, Wu
peiyuanwu@ntu.edu.tw

Chiao-Yun, Chin b10901154@ntu.edu.tw
Yi-Chen, Wang b10901143@ntu.edu.tw
Yi-Chou, Lin b10901149@ntu.edu.tw

RELEASE DATE: 04/03/2025
DUE DATE: 04/17/2025

Late policy: $Score = Score \times 0.7^{\lceil h \rceil / 24}$, $h = \text{late hours} \leq 48$

1 Problem Description

A city planner wants to find the largest rectangular piece of land available for constructing a new building. The city is represented as a $m \times n$ grid, where each cell is either empty (1) or an occupied (0). Your task is to determine the largest rectangular area consisting only of empty plots(1s) and return its area.

For example, given the following city map: $\begin{bmatrix} 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$ (as shown in Fig. 1). The largest available rectangular land for is in rows 2–4 and columns 3–4, resulting in an area of 6.

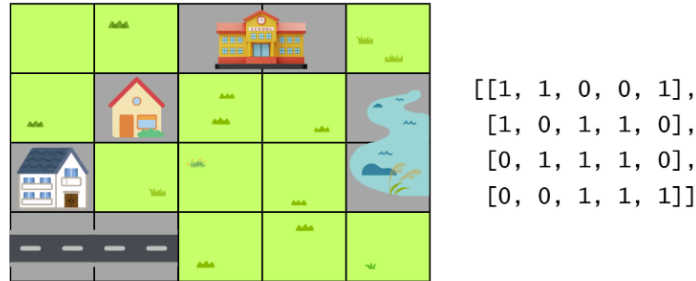


Figure 1: 2D city map

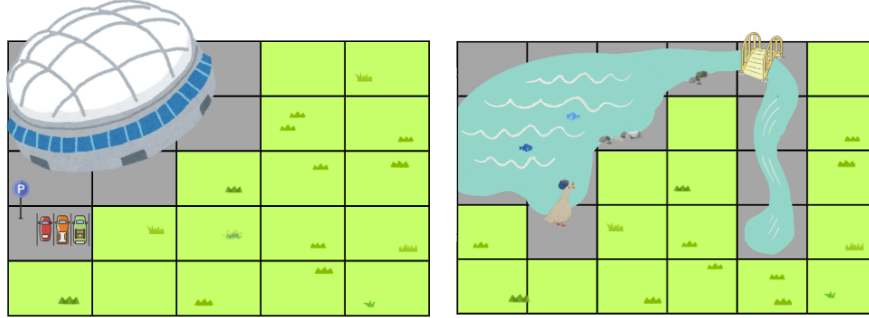


Figure 2: An example for 1D increasing version Figure 3: An example for 1D general version

2 Implementation Requirements

We provide the templates for **Ver_1D_Increase**, **Ver_1D_general**, and **Ver_2D** in **MaxPlace.py**. Your task is to complete the sections marked as **TODO**.

2.1 1D Increasing Version(40%)

To simplify the problem, let's first consider a "one-dimensional increasing version", where the problem is simplified and the city is represented in one dimension. Given an array of numbers representing land heights that increase from left to right and determine the largest rectangular area that can be formed.

For example, given the increasing heights: [1, 2, 3, 5, 5], as shown in Fig. 2. The largest rectangular area is 10, covering columns 4 and 5 with height 5. In **MaxPlace.py**, there is a function named **Ver_1D_increase**. Your task is to finish the code and return the maximal rectangle area in time complexity $O(n)$.

- The input of the function is an array of length n ($A[0, \dots, n-1]$).
- Each element in the array is non-negative integer and the array is non-decreasing ($A[i] \leq A[j], \forall i < j$).
- The function should run in $O(n)$ to get full scores of time complexity.

2.2 General 1D Version(30%)

Next, extend the solution to a general one-dimensional version where heights are not necessarily increasing.

For example, given the heights: [2,1,3,4,1,4], as shown in Fig. 3. The largest rectangular area that can be formed is 6, covering columns 3 and 4 with height 3 or columns 1 to 6 with height 1.

There is a function named **Ver_1D_general**. You should finish the code according with the following requirement.

- Input: an array of length n , each element in the array is non-negative integer.
- The function should run in $O(n)$ time complexity to receive full points.
- Hint: Can change the whole problem into multiple increasing segments.

2.3 2D City(30%)

Finally, we can extend the solution to handle the “two-dimensional city grid”. Same as the problem description said the function would receive a $m \times n$ list and should return the max empty rectangular area. The function **Ver_2D** in **MaxPlace.py** should be completed according to the following requirements:

- The input of the function is a 2-dimension list with size $m \times n$, each item in the list is either 0(occupied) or 1(empty).
- The function should run in $O(mn)$ to get the full timing score.
- Hint: Treat each row as a base and compute the connected empty areas as heights, reducing the problem to m one-dimensional problems. Then you can use **Ver_1D_general** to solve the problem.

3 Evaluation

Use the commands “python3 testcase1.py”, “python3 testcase2.py” and “python3 testcase3.py” to verify the correctness of **Ver_1D_increase**, **Ver_1D_general**, and **Ver_2D** respectively. Comparing your result with golden_1.txt, golden_2.txt, and golden_3.txt to check your code.

Note:

1. Do not modify the interface of the functions, but you can add your own functions.
2. You may allow use Numpy but other statistics libraries are not allowed.
3. The time complexity of **Ver_1D_increase** and **Ver_1D_general** should be $O(n)$, and **Ver_2D** should be $O(mn)$. where n is the length of array and $m \times n$ is the size of two dimension list.

4 Submission

Please put maxPlace.py into a directory named studentID and compress the directory into studentID.zip to NTUCOOL. The homework is due on 04/17, at 23:59

5 Grading policy

This programming assignment will be graded based on correctness and time complexity. Public test cases: `testcase1.py`, `testcase2.py` and `testcase3.py` and their corresponding golden output: `golden1.txt`, `golden2.txt` and `golden3.txt` are provided, but additional private test cases will be used for evaluation.

- **1D Increasing Version:** 40%
- **General 1D Version:** 30%
- **2D City Grid:** 30%

If your program passes all test cases, you receive 60% of the points for each section. If it also meets the required time complexity, you receive the remaining 40%, for a total of 100 points.